

MECHENG 706

Mechatronics Design - Case Studies

Lecture 2, Week 7



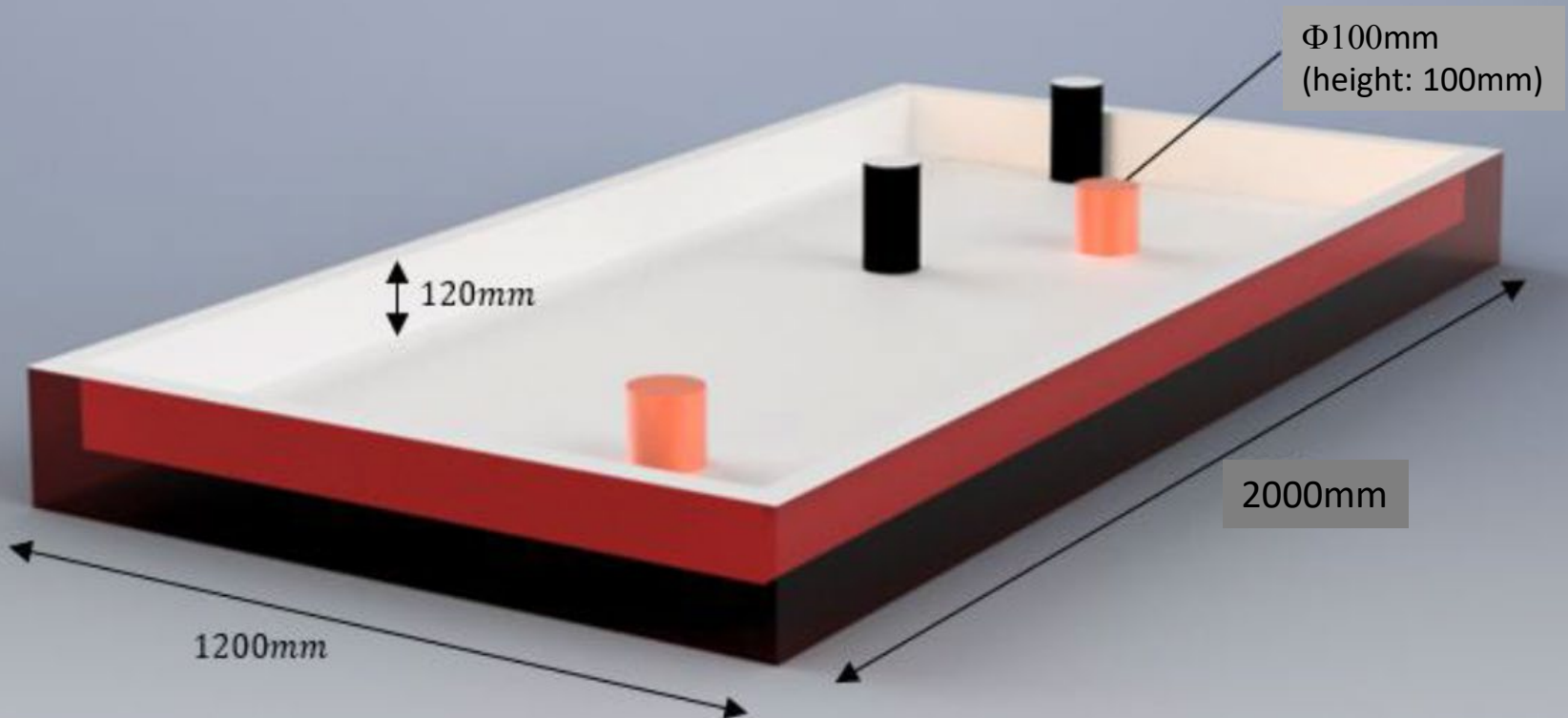
THE UNIVERSITY OF
AUCKLAND
Te Whare Wananga o Tamaki Makaurau
NEW ZEALAND

ENGINEERING

1. Overall Design Considerations

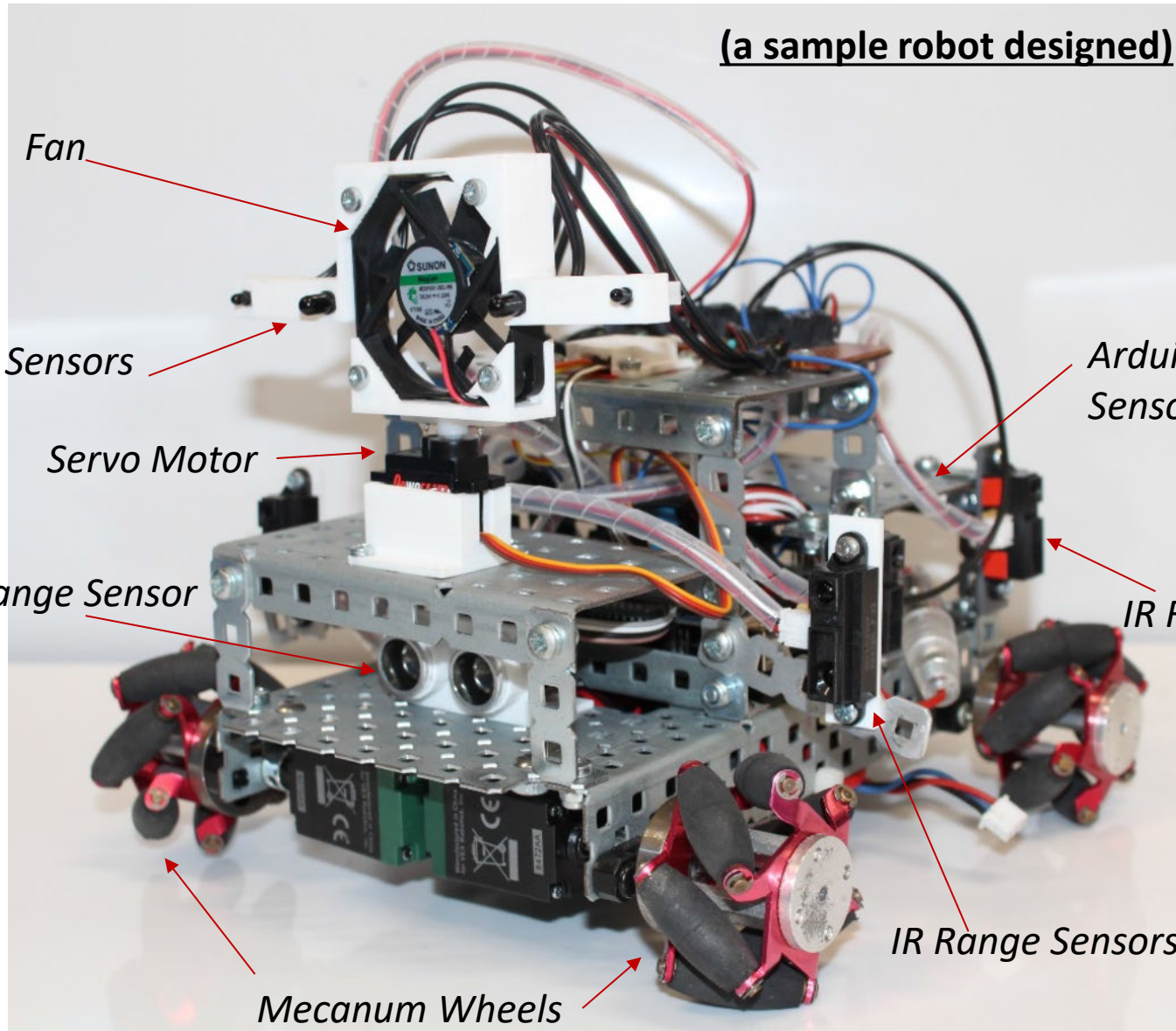
First step of the mechatronics design – you will need to perform analysis of what is required in what environment or scenarios, and understand what is available to get it started from. (This is part of the Agile/sprint planning.)

Arena, obstacles and fires



The challenge (the 2018 Design Project)

SLAM, Area Covering, Navigation, Obstacle Avoidance and Firefighting



A Sample Robot at Work

(SLAM, Area Covering, Navigation, Obstacle Avoidance and Firefighting)



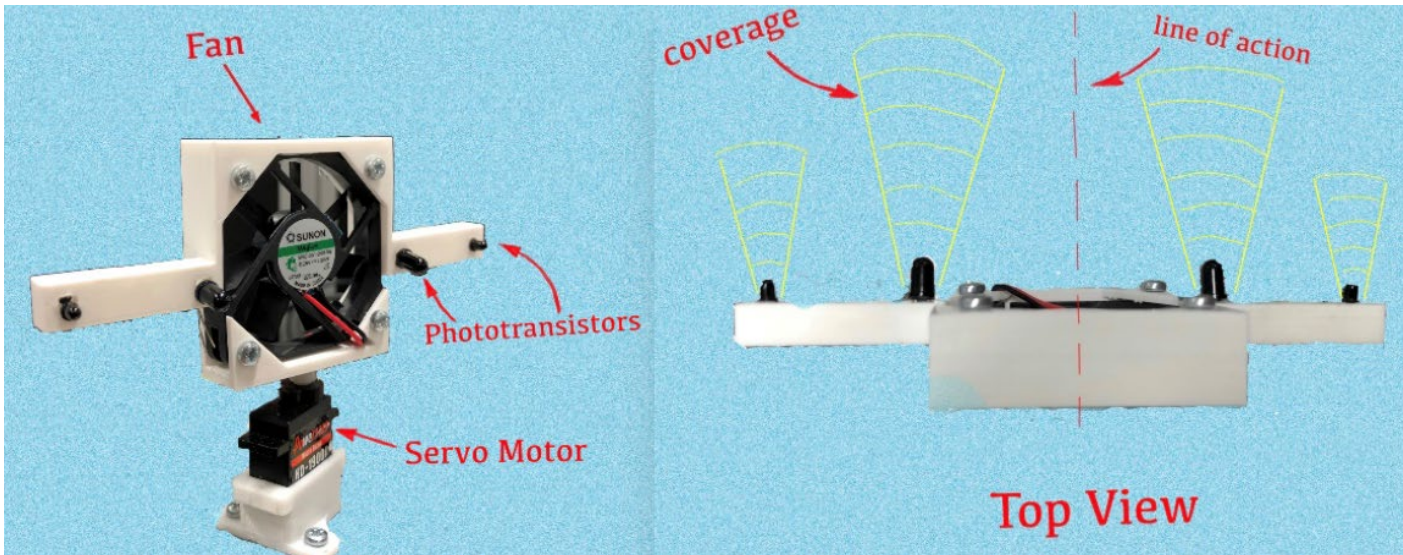
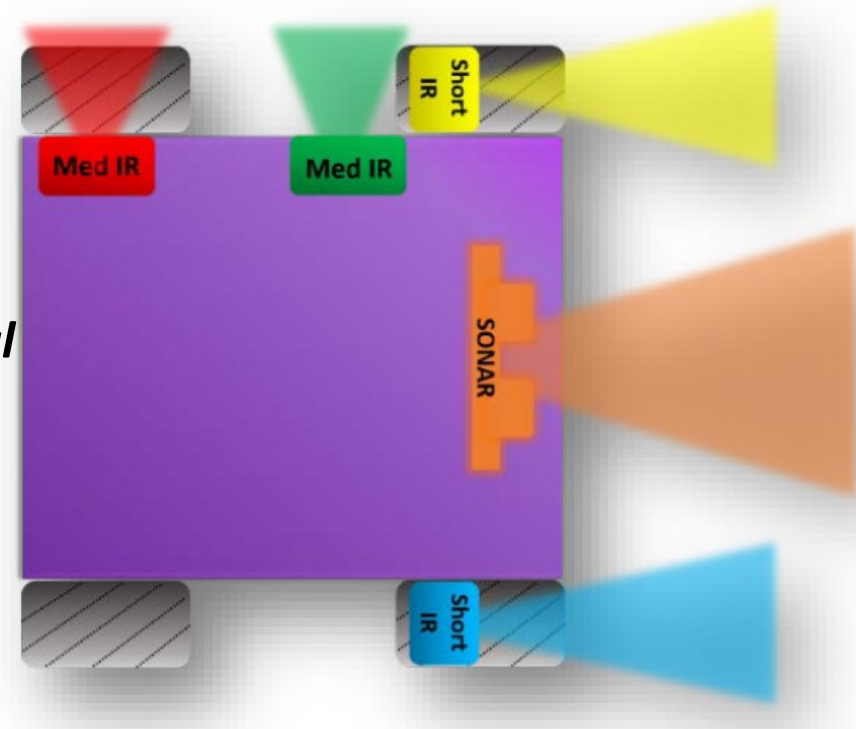
This video shows basically what you will need to do with your Project 1 and 2 in combination.

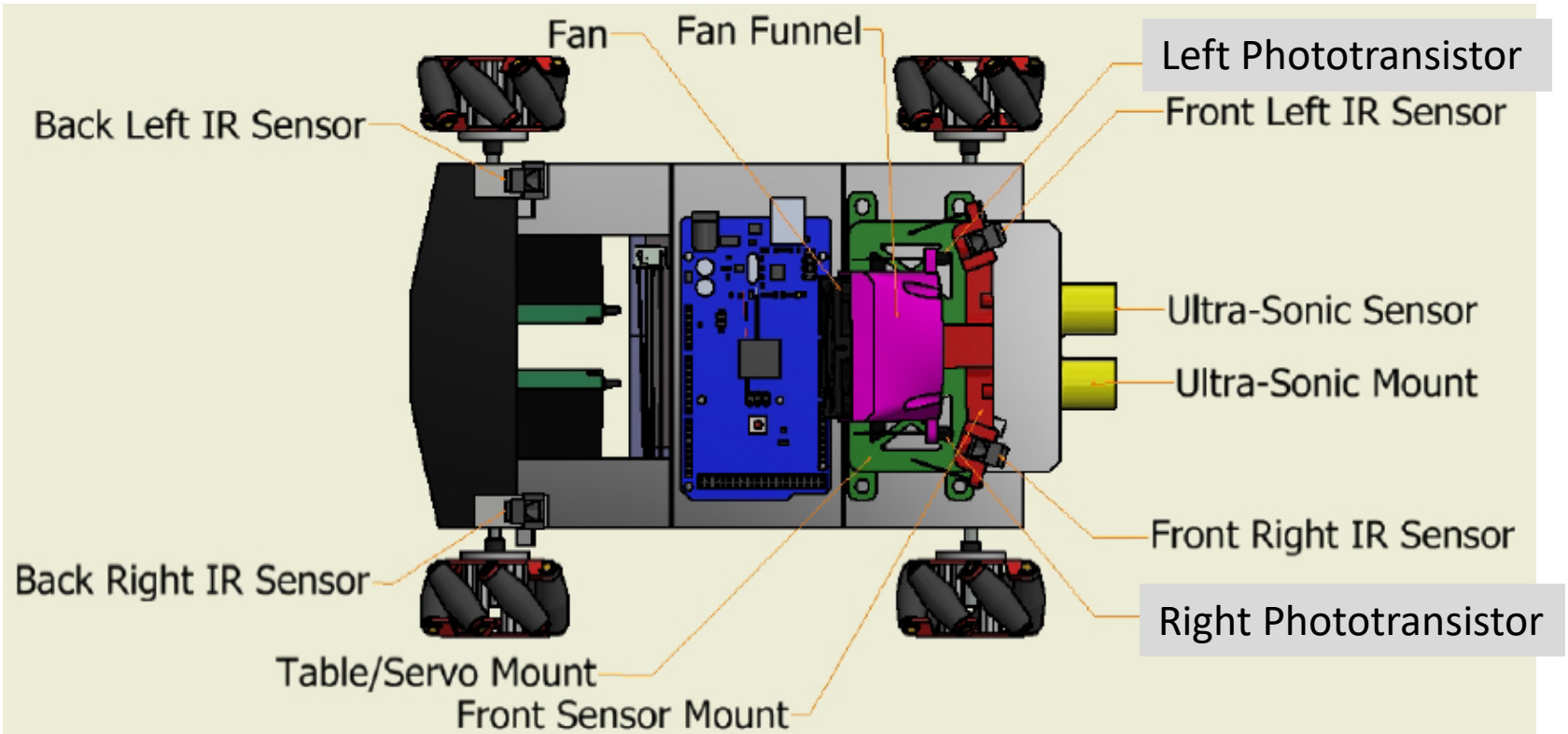
<https://www.youtube.com/watch?v=3hRZsSBI39U>

(the 2013 design challenge: Autonomous Navigation/Obstacle Avoidance/Firefighting)

Placement of range and fire sensors

- 4 walls can be used as reference for localisation and navigation of the robot.
- The spacing gap between two IR range sensors – ***the larger the better!***
- IR range sensors are placed ***parallel/vertical*** to the surface to be measured.
- Sonar range sensors for long range.
(Sensing design for the closed-loop control)
- Phototransistors as a fire sensor on servo motor?
- Fan as a fire extinguisher on servo motor?





Another design (with different sensors placement):

- the back short range IR sensors were dedicated to point behind the robot.
- the purpose of the back pointing sensors is to align with the back wall which is crucial in implementing the navigational strategy zig zag path the robot will follow.
- the ultrasonic sensor mounted below the chassis.
- the ultrasonic sensor funnels narrow its field of view, so as to not give false readings.
- the medium range IR sensor and fan on the servo that rotates 180 on the front face.
- The fan funnel forces most of the flow to travel straight ahead.
- Phototransistors as a fire sensor are mounted on either side of the fan mount.

2. System Architecture

Having discussed the placement of sensors/fan on the robot and the robot's mobility, we now need to perform **HW/SW integration of subsystems** in order to deliver the robot intelligence (i.e, SLAM/Area Coverage/Avoidance/Firefighting)

First we will need to understand how the subsystems **are connected/interfaced to the microcontroller**, for functionalities we specify/want the robot to achieve.

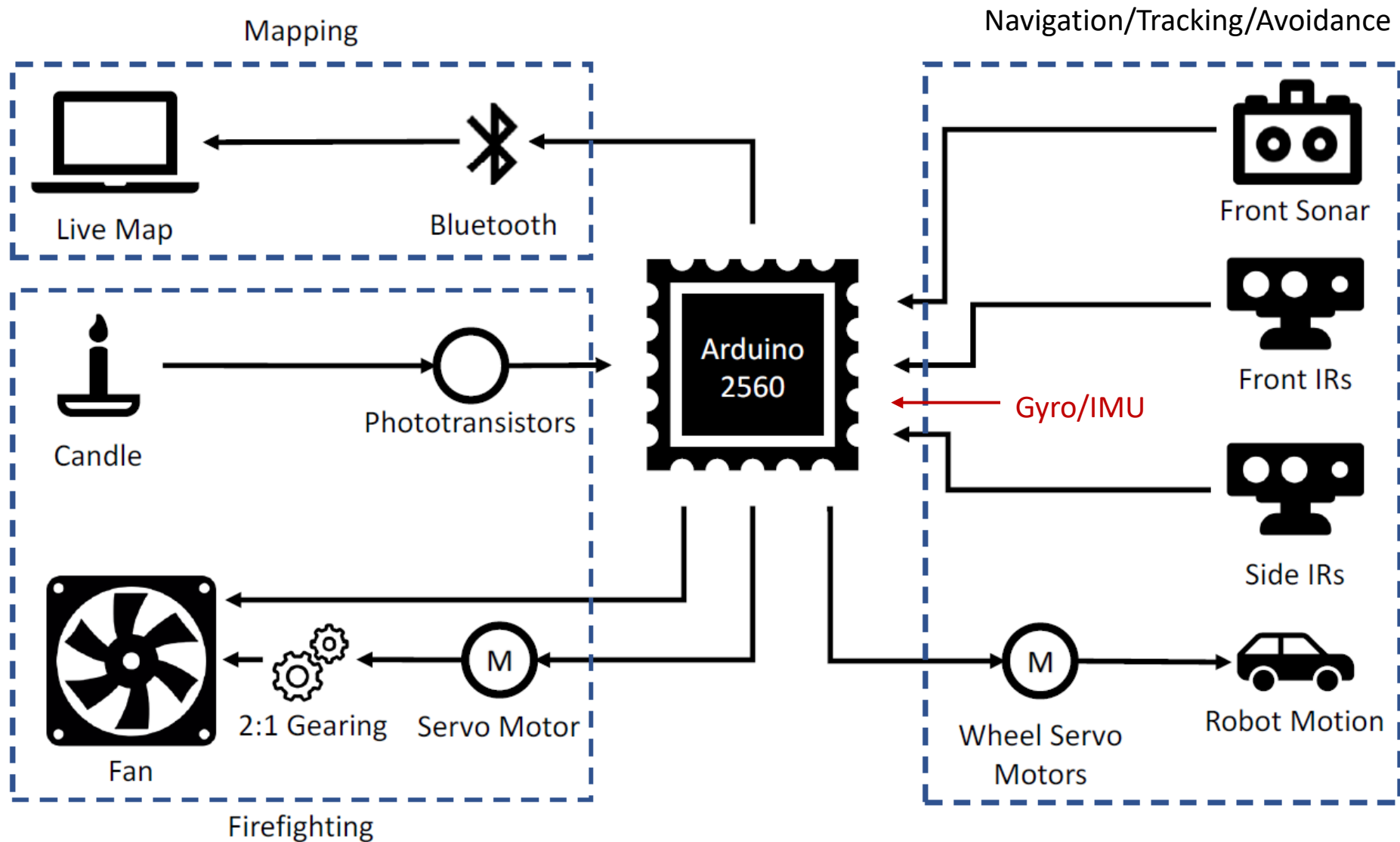
To get the design started, we begin with overall design, **architecture and high-level** functionalities of the robot.

- ***Schematic/generic architecture***
- ***Detailed design*** showing pin connections, one by one, and stating pin functions
- *Both will be needed in the project report.*

By now half the project has been completed by the provision/selection of sensors, motors, components and mobility platform, and the suggestion of their placements.

We will need to continue to connect parts to microcontroller and decide which pins for what parts and what purposes. And write codes to make the sensors/actuators working and then deliver the robot intelligence.

System architecture in viewpoint of hardware integration



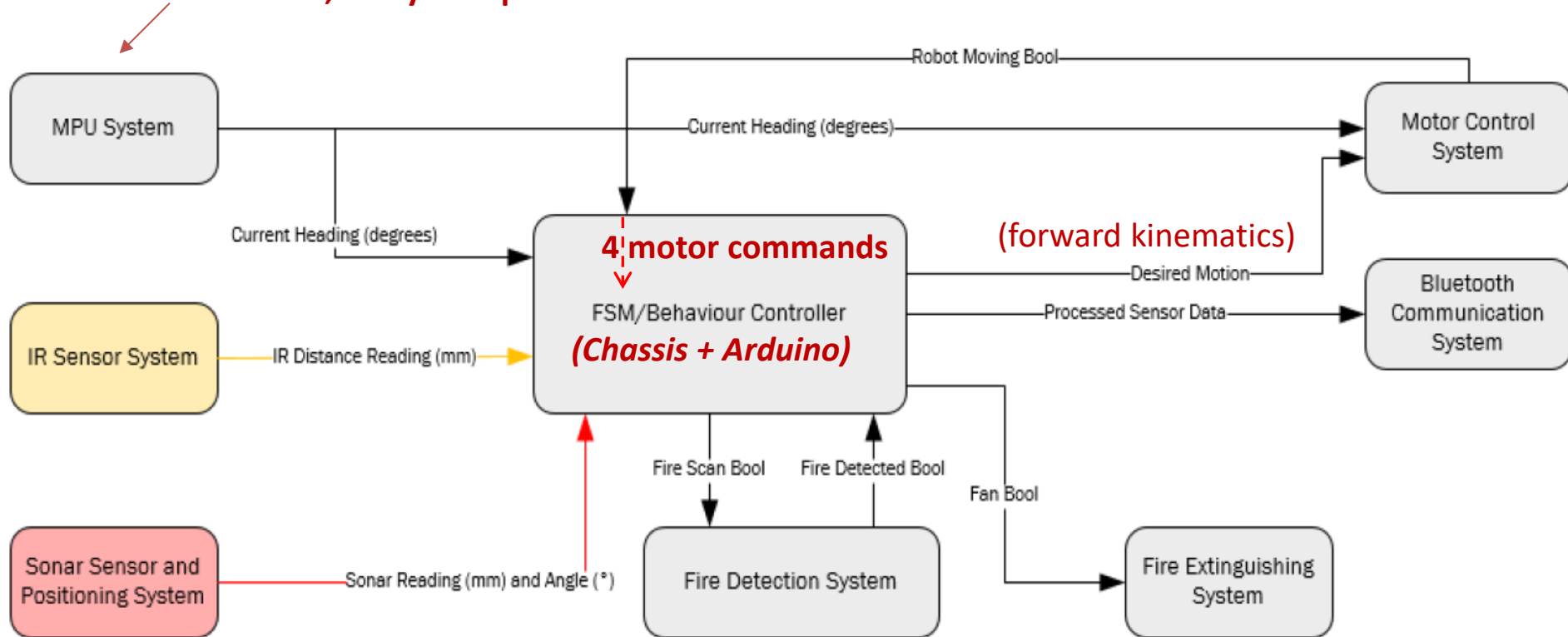
Microcontroller's Pin Allocation

Pin No.	Input/ Output	Devices (sensors/ Actuators)	Type of signal	Function	Symbol	...
A0	Input	IR Range sensor 1	Analog	Range 10~40mm	d_front	
A1	Input	IR Range sensor 2	Analog	Range 10~80mm	d_slide	
...	...	...	...	...	...	
48	Output	Servo motor	Digital	Positioning (0~180°)	servo_fan	
...	...	...	...	...	...	

(incomplete, more such as motors being interfaced to MCU needs to be added.)

System architecture in viewpoint of functional diagram

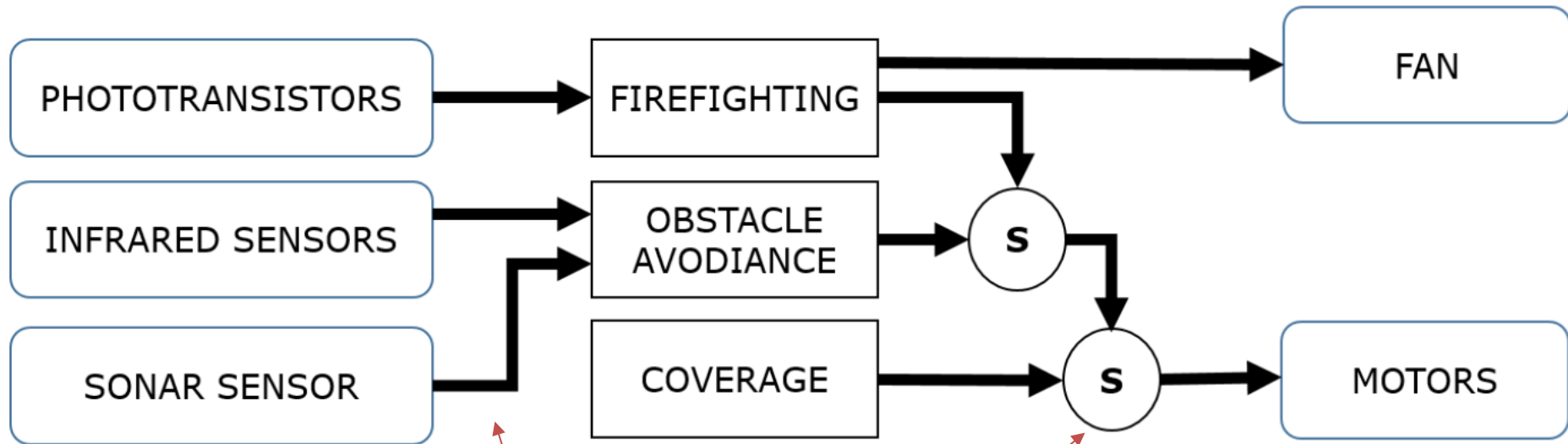
MPU motion sensor, or Gyroscope



Robot intelligence (hosted at the MCU) – SW integration in the middle block. We need to be clear about what's the inputs and what's the outputs.

Robot intelligence is implemented in codes run by microcontroller. (e.g., BC, FLC, ANN methods) Four motor commands are the output of the controller and applied to the robot via MCU/motor drive.

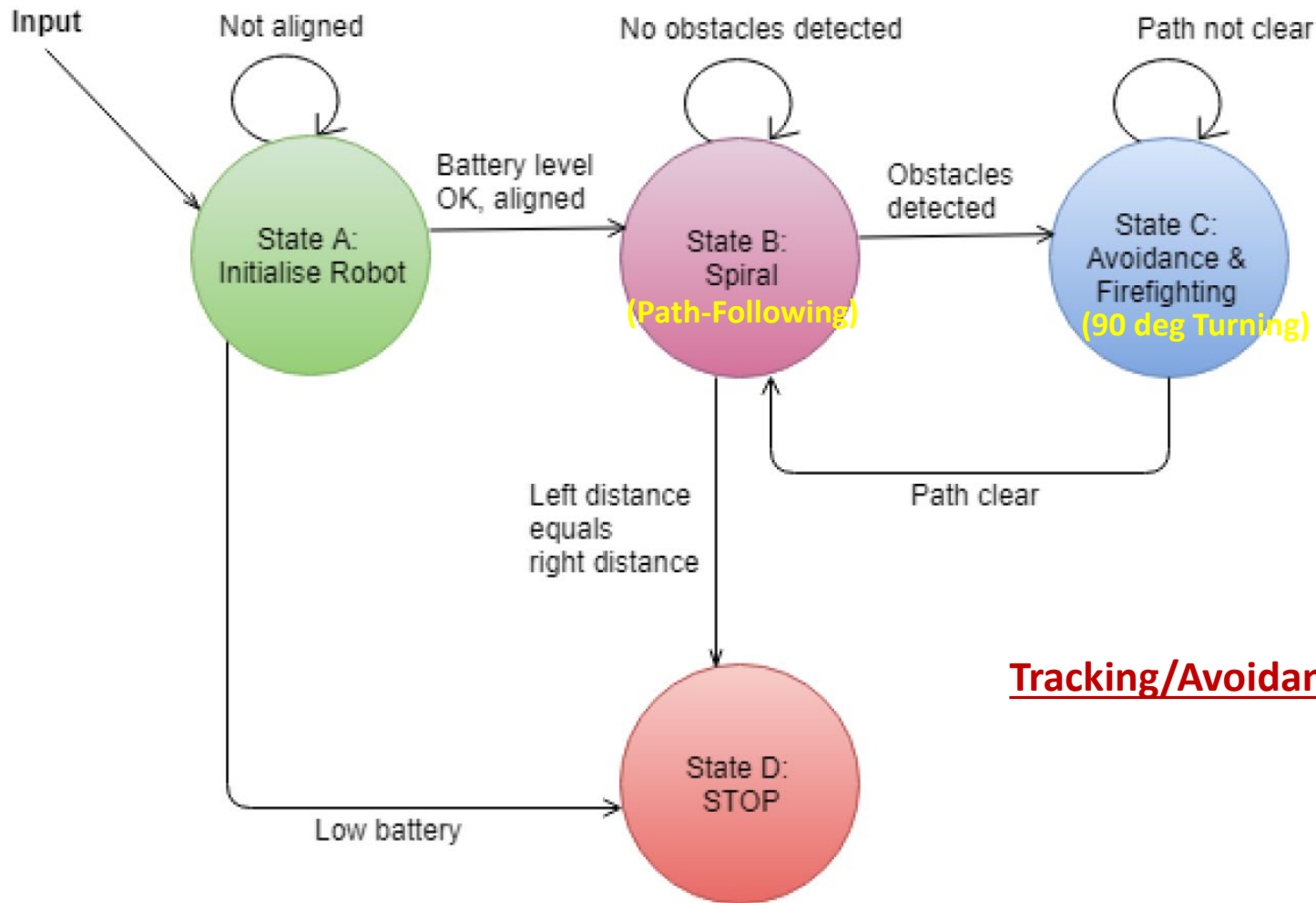
System architecture in viewpoint of behaviour network



**S nodes stand for fusion of single simple behaviours.
No sensor fusion is required.**

At the core of the functional diagram is Robot behaviour, or Behaviour network that we will learn in Week 8. Each behaviour is simple, a reactive finite state machine (FSM).

System architecture in viewpoint of finite state machines



Tracking/Avoidance Robot

Robot intelligence is described in terms of robot motions at various situations/scenarios. In this example, there are two types of motions, one being Path Following and the other being Turning. The robot intelligence is defined by a finite state machine of a few robot motion states.

In summary, for a good design, we will need to present our overall design in number of ways, showing different aspects of the design. This is expected in the project report.

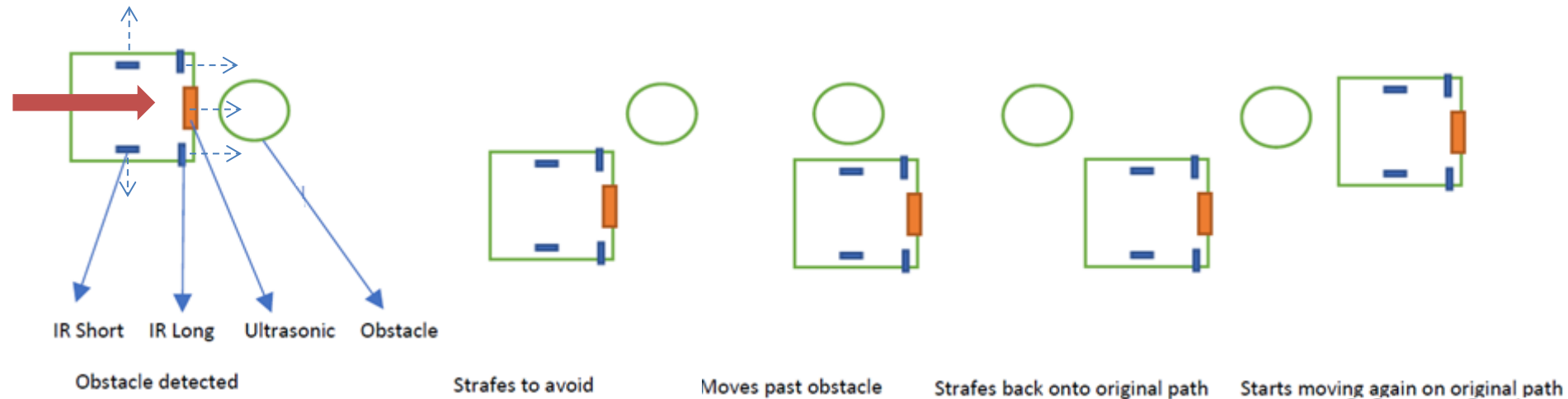
3. Sensing/Actuation for Obstacle Avoidance

*Obstacle Avoidance System or behaviour is part of overall robot intelligence and requires both sensing and actions of the robot. **a mechatronic subsystem!***

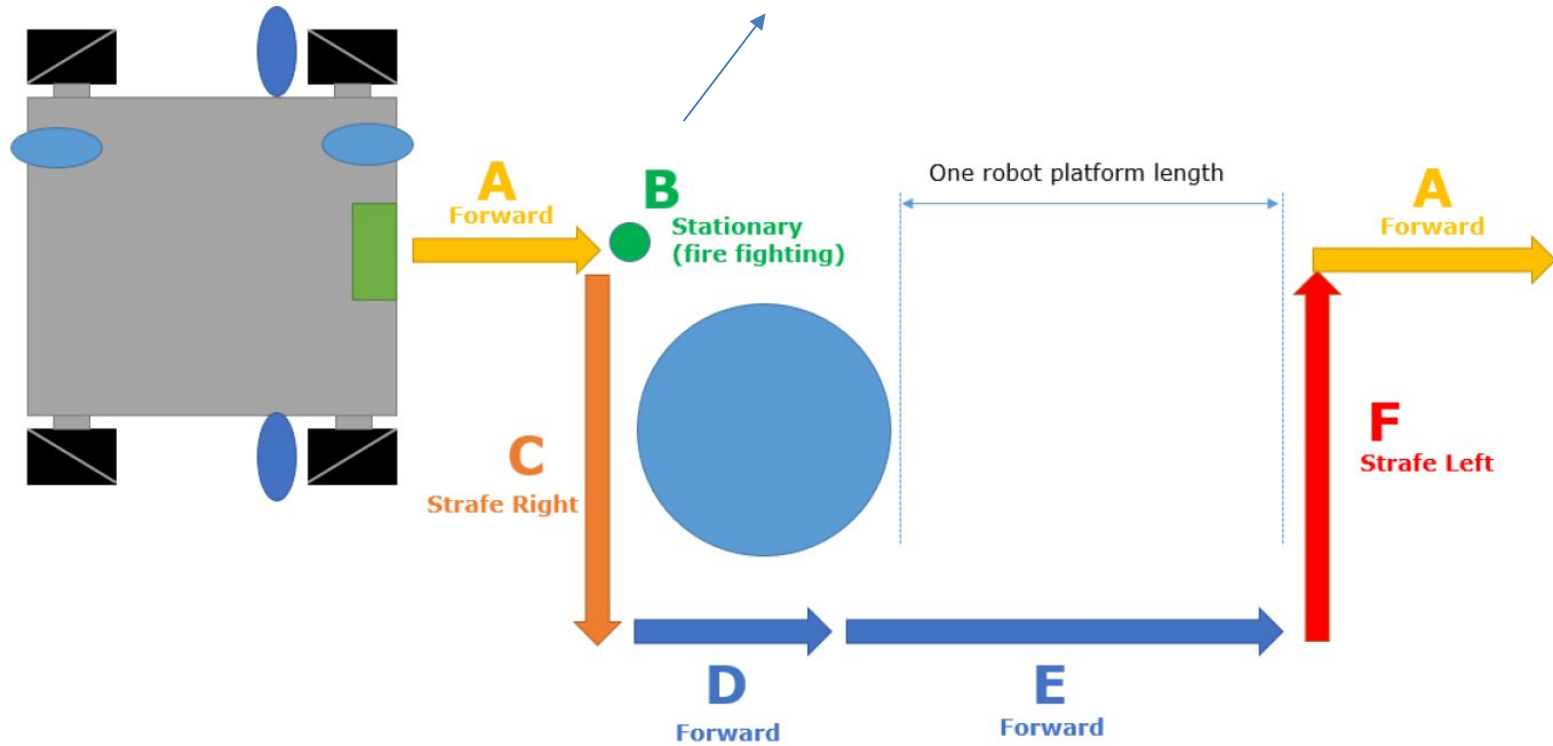
Design 1

What the subsystem does:

1. To detect obstacles and carry out motion to dodge them while travelling.
2. Each obstacle detected triggers a state or behaviour which must lead to avoidance without collision into obstacles.
3. This is implemented as a subsystem of the overall structure of the robot intelligence.



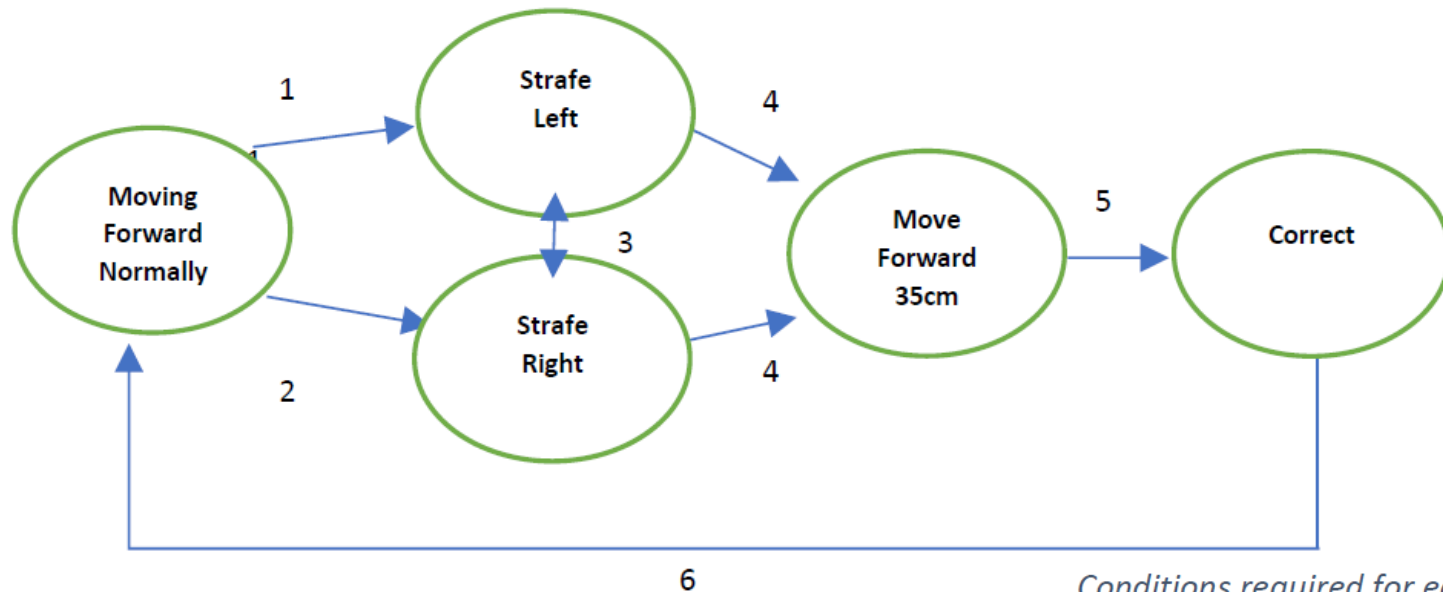
If the obstacle detected is a fire



How the subsystem was designed:

1. The sensors calibrated/used were: 1 ultrasonic sensor, 2 long range IR sensors, and 2 short range IR sensors, which were placed as depicted in the last slide.
2. The short range IR sensors were calibrated as *a linear function of $1/d$* ,
$$d = \frac{13}{0.0049 V}, cm$$
3. The long range IR sensors used *a look-up table* that relates analog voltage to distance. *(it could also be a linear function of $1/d$.)*
4. The sonar used the time of flight between a pulse trigger and the echo which is converted using the speed of sound.
5. The detection and avoidance were implemented via a finite state machine.

State Machine for Obstacle Avoidance



Conditions required for each state change

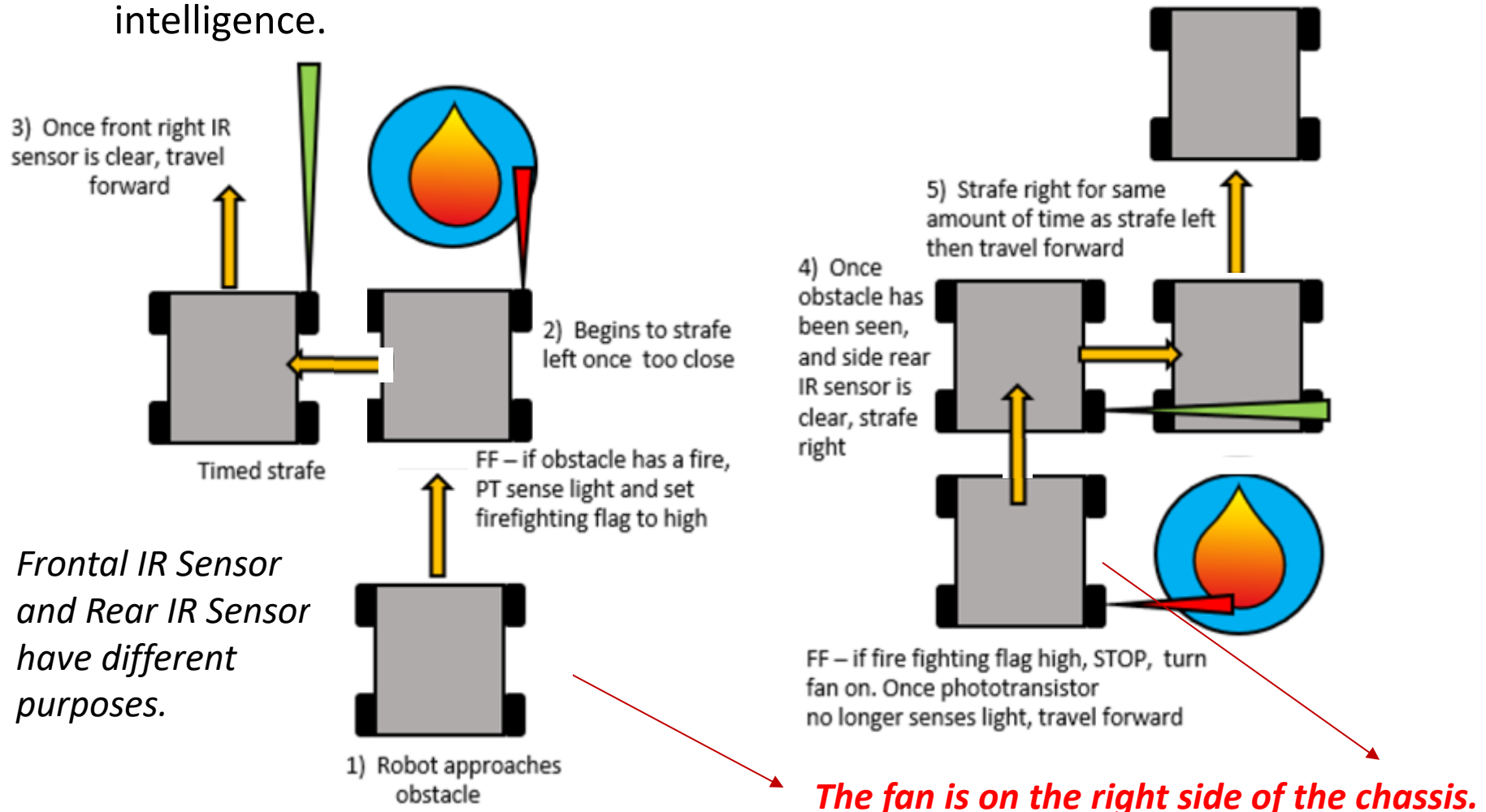
A FSM consists of a number of ***motion states*** of the robot, which can be transitioned between them.

	Condition for state change
1	Left front IR sensor detected obstacle
2	Sonar or right front IR sensor detected obstacle
3	Obstacle encountered while strafing sideways
4	Obstacle no longer being detected
5	Have moved 35cm and so have gone past obstacle
6	Strafed back same time it took to strafe out

Design 2

What the subsystem does:

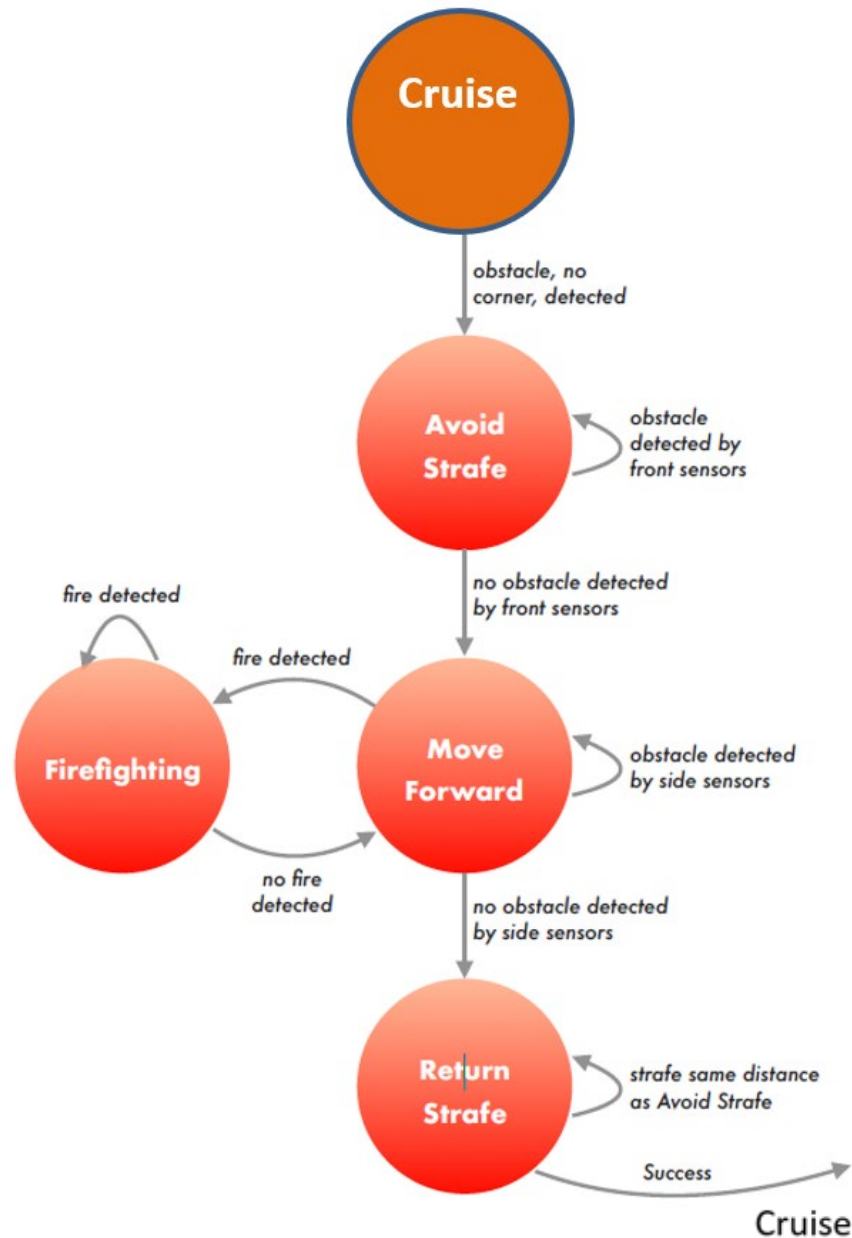
1. To detect obstacles and carry out motion to dodge them while travelling.
2. Each obstacle detected triggers a state which must lead to avoidance without collision into obstacles.
3. This is implemented as a subsystem of the overall structure of the robot intelligence.



How the subsystem was designed:

1. One medium range IR sensor and one short range IR sensor are located at the front and rear of the robot to detect the obstacles, respectively.
2. The robot travels forward in a straight line until it detects an obstacle within 6 cm (*edge-to-edge*).
3. Depending on which sensor detects the obstacle in the front, the robot strafes left (or right) for an appropriate period of time to clear the obstacle.
4. Once the robot has travelled far enough to the left to avoid colliding with obstacle, it travels forward again, for a period of time long enough to clear the obstacle.
5. Once the robot is far enough forward, it strafes to the right the same amount of time it strafed to the left. This ensures the robot returns to its original path.
6. All of IR sensor values were filtered using *a simple moving average filter* to smooth out any noise before the values were used to determine the robot's movement.

How the robot behaviour is described – FSM



AVOIDSTRAFE makes the robot strafe to the left or right, depending on the position of the obstacle relative to the robot, and whether or not it is a fire. If the obstacle is a fire, or the obstacle is detected during *LOOP0*, the robot always strafes to the left, this is **due to the position of the fan on the right side of the robot**, and to avoid collision with any of the arena walls. For all other cases, the robot strafes in the shortest direction.

A boolean is used to ensure the robot has strafed far enough to clear the obstacle entirely, ensuring no collision occurs, then *MOVEFORWARD* is returned. While the robot is strafing, its strafe distance is tracked and is used later in the *RETURNSTRAFE* state.

MOVEFORWARD causes the robot to travel forwards until the fan becomes level with a potential fire. The phototransistor is then checked to determine if it is a regular obstacle or a fire. If a fire is present **FIREFIGHTING** is returned, otherwise the robot continues forwards until it has cleared the obstacle then returns *RETURNSTRAFE*.

RETURNSTRAFE uses the strafe distance saved in *AVOIDSTRAFE*, to strafe the correct distance back to its original path then returns the appropriate loop state, depending on the value of *loops*.

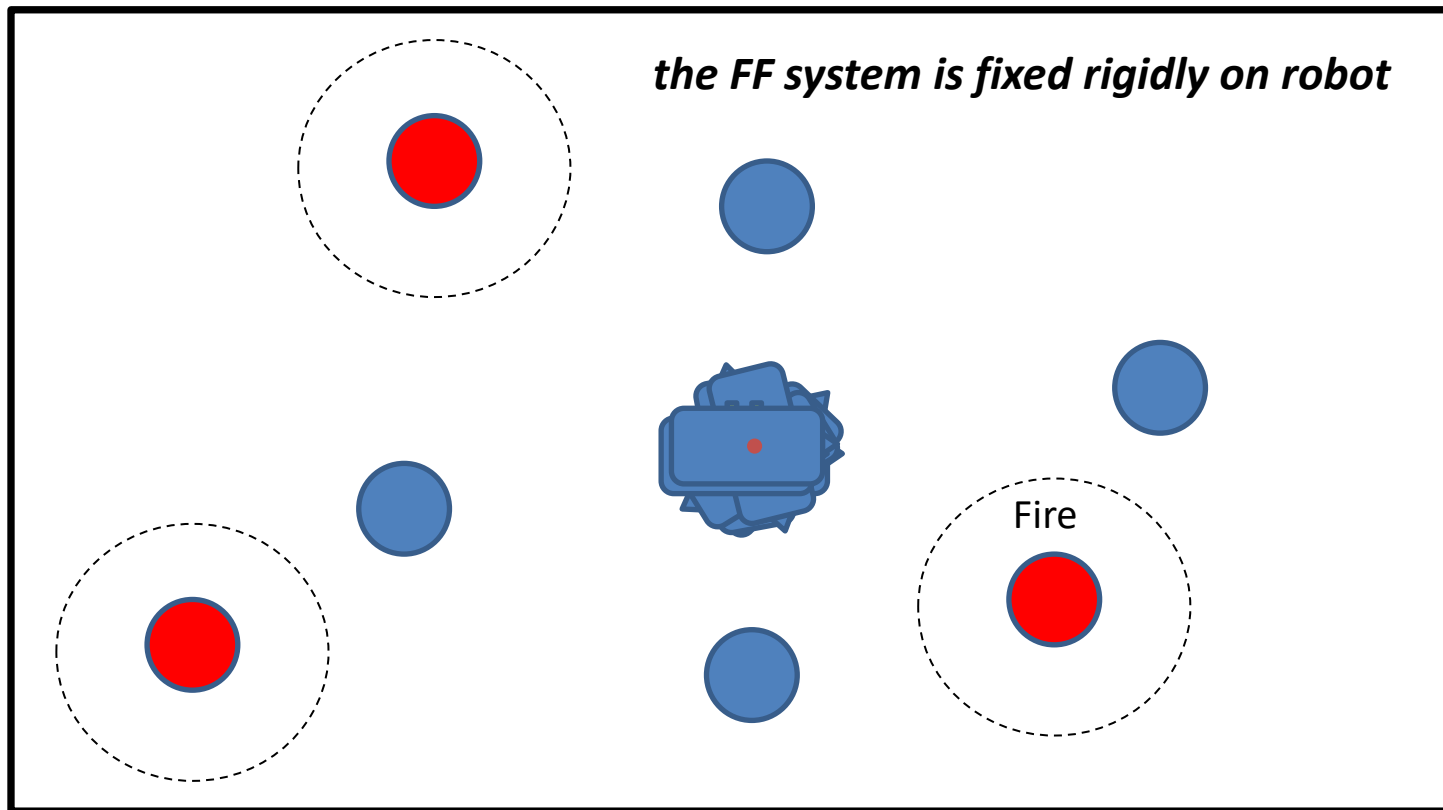
4. Sensing & Actuation for Fire Detection /Extinguishing

The Fire-fighting (FF) system is a mechatronic subsystem:

** Phototransistors for fire detection * a fan for fire extinguishing * real-time codes for behaviour*

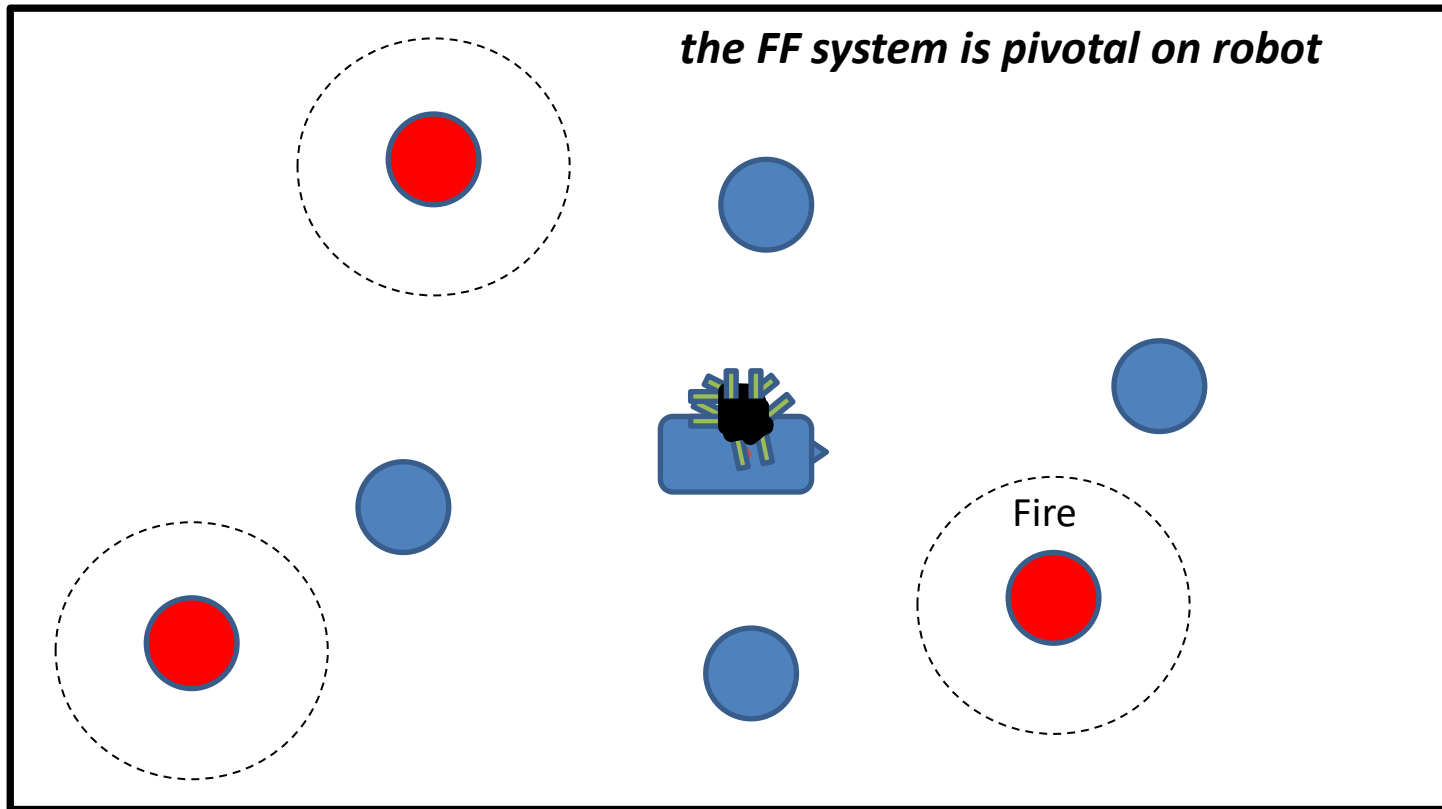
Task 1: design of a FF system and Task 2: placement of the FF system

Conceptual Design 1



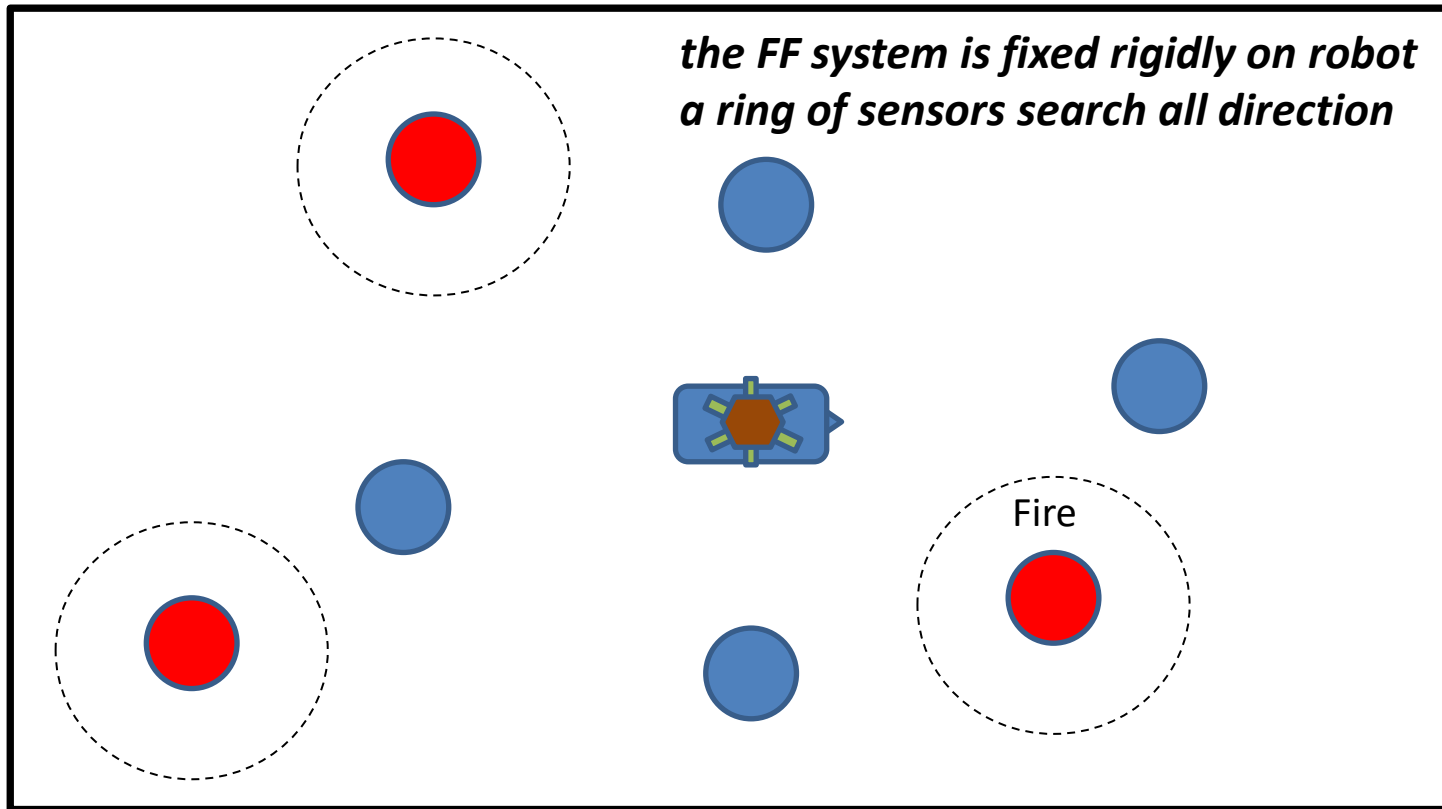
The robot/FF system turns together on spot

Conceptual Design 2



The FF system rotates while the robot remains stationary

Conceptual Design 3



A ring of sensors is used while the robot remains stationary.
Neither of the FF system and the robot rotate when searching for a fire.
Once a fire is found, the robot/fan turns and moves toward the fire.

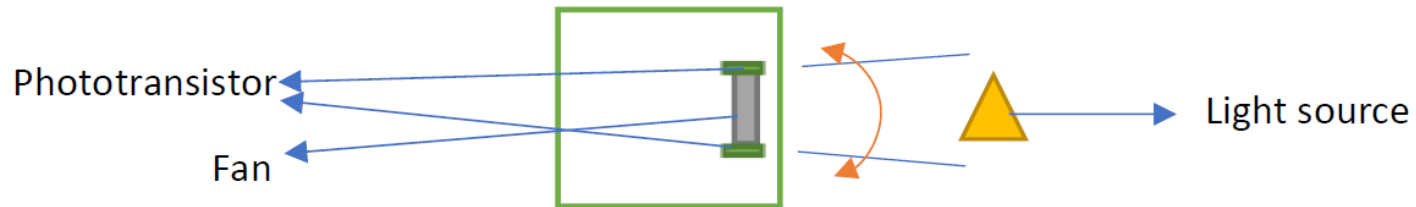
Design 1 (Simple)

What the subsystem does:

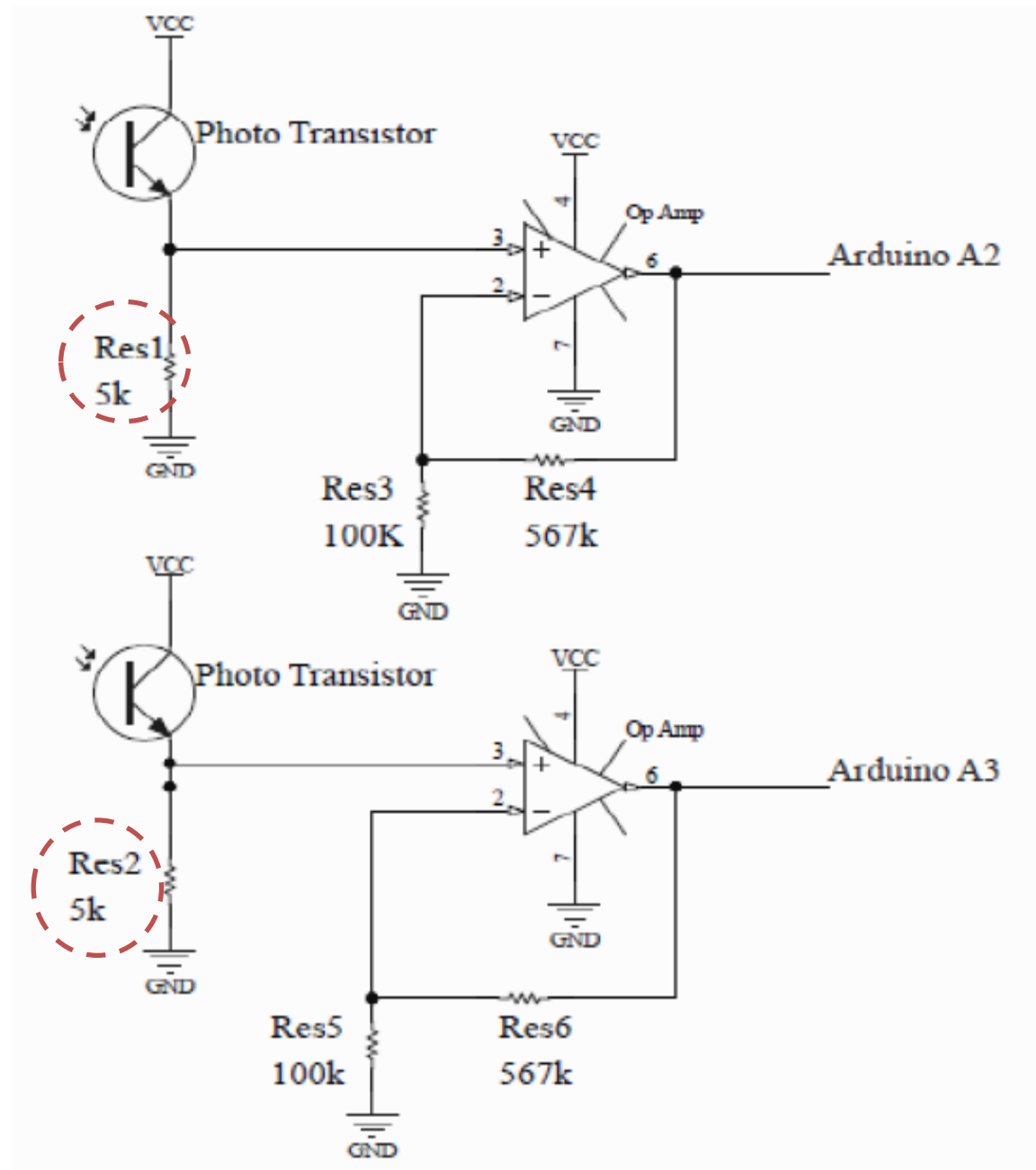
1. To detect and extinguish a fire placed on top of an obstacle.
2. The fire is in the form of a visible LED with a thermistor turning it off.
3. It is detected with phototransistors and then extinguished by a fan with the robot being within a centre-to-centre distance of **20** cm.

How the subsystem was designed:

1. Two phototransistors were used for fire detection. They were positioned on either side of the fan.



2. The fan was positioned at the front of the robot and mounted on a servo motor. This was to allow for the fan to rotate towards the fire if the fire was not directly in the front of the robot.
3. As the readings of the two phototransistors exceed a certain threshold, the fan is then turned on through the switching circuit.

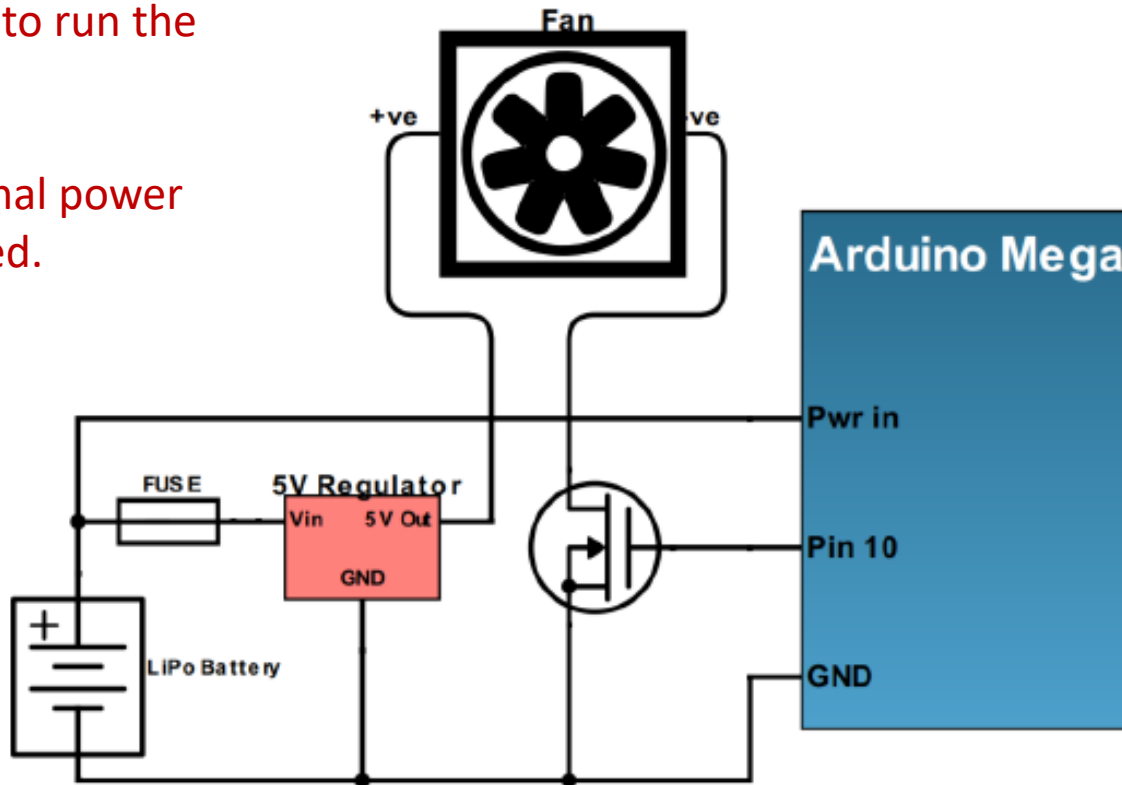


0.3 A current is required to run the fan.

An external power is required.

Vdd can provide only 0.2 A current.

An output pin can provide 0.02 A current.



The fan could not be connected directly to the Arduino board and an external power electronics circuitry was required due to maximum current limitations.

Fan nominal current draw = $\text{Power}/\text{Voltage} = 1.45\text{w}/4.8\text{v} = 0.3\text{A}$

Arduino pin maximum current source = 0.02A

Arduino VDD maximum current source = 0.2A

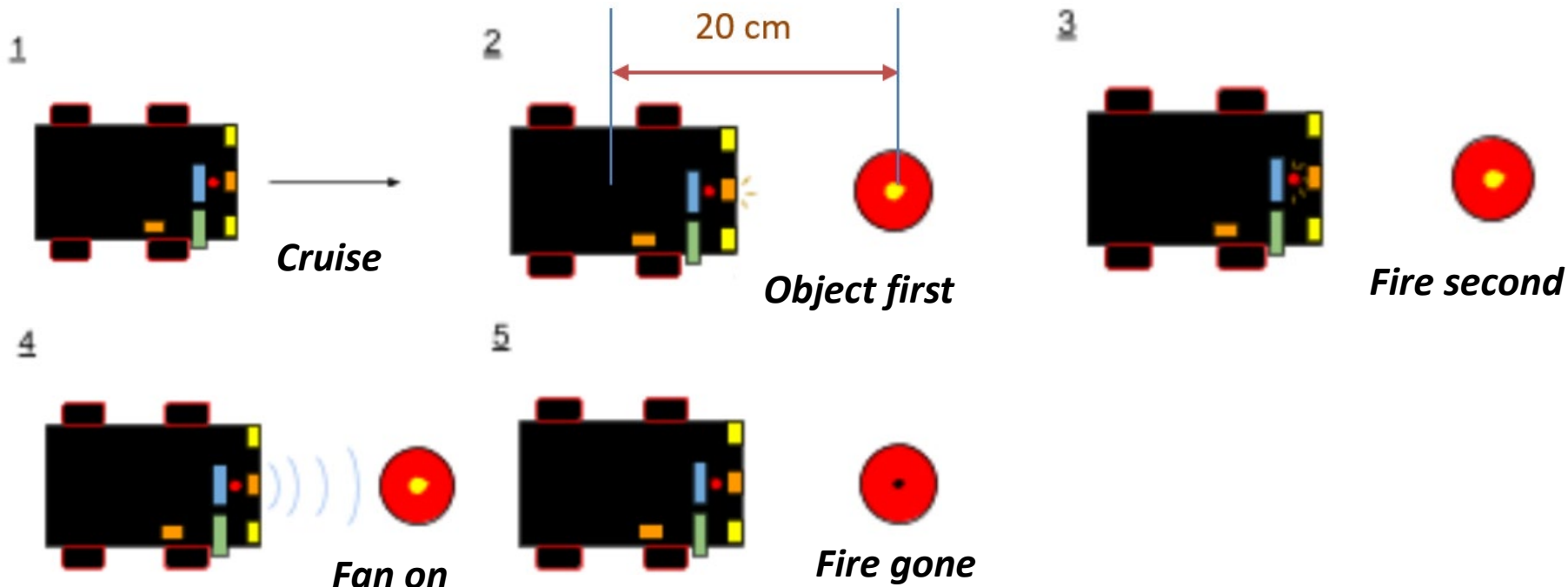
The solution was to draw current directly from the 7.4V battery using an Arduino pin as an enable switch. This was achieved using a 5v regulator IC in conjunction with a power MOSFET.

Here the regulator steps the battery voltage down for use with the fan, and is switched enabled/disabled by switching the MOSFET gate pin with an Arduino output pin.

Design 2 (Easy)

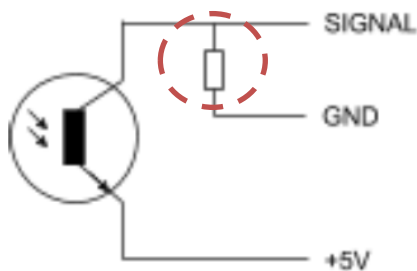
What the subsystem does:

1. The robot moves forward until the range sensor senses a fire object **20 cm** in front of it.
2. The robot comes to a complete stop.
3. A electric candle is detected by the phototransistor.
4. The fan turns on until the candle is blown out.
5. The fan turns off when the phototransistor can no longer sense a light.

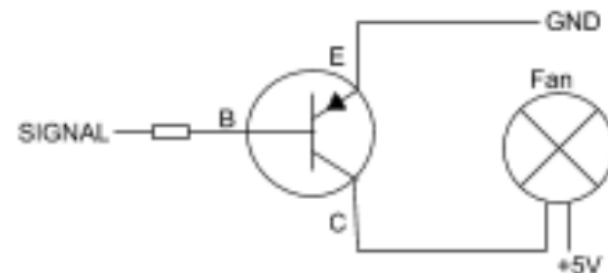


How the subsystem was designed:

1. Initially, the robot travels forward until the medium range sensor **senses an object 20** cm from it.
2. A reading is taken from the phototransistor circuit using the Arduino function **analogRead()**. This function returns an integer value between 0 and 1023, with 0 being 0 volts and 1023 being 5 volts. When no candle, the readings are below 200 (~1V). When a candle present, the reading will rapidly increase to 900-1000 (~4-5V).
3. Very often, the sensor readings from the phototransistors are noisy, and a simple **Moving Average filter** was implemented in Arduino, by taking the **average of 10 continuous samples**.
4. The fan was operated by the use of a bipolar junction transistor. The BJT used is a BC327 **PNP transistor** which effectively makes it an active low component. In this case, a LOW signal will allow current to pass from the emitter to the collector of the BJT and hence produce a voltage across the fans.
5. In the setup() loop, the fan pin is given **a signal of HIGH**, because the default signal for any Arduino pin is LOW. Hence, this fan setup is crucial to avoid turning on the fan when it has not yet detected a candle.



Circuit diagram of the phototransistor



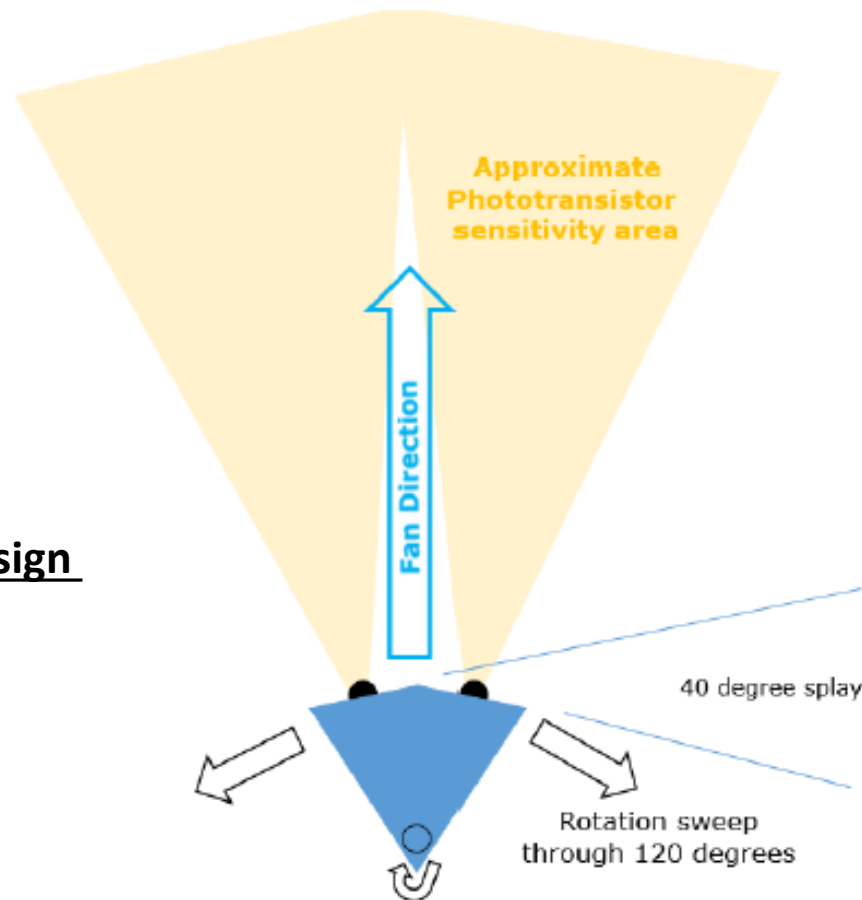
Circuit diagram of the fan

Design 3 (Advanced)

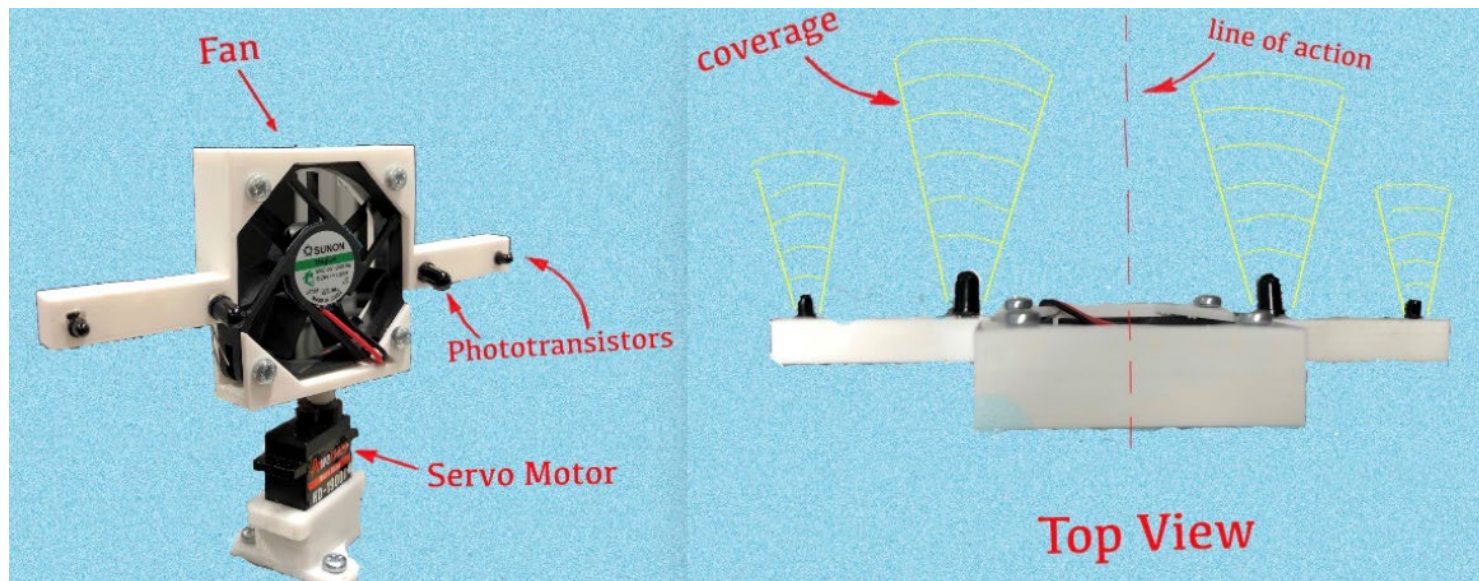
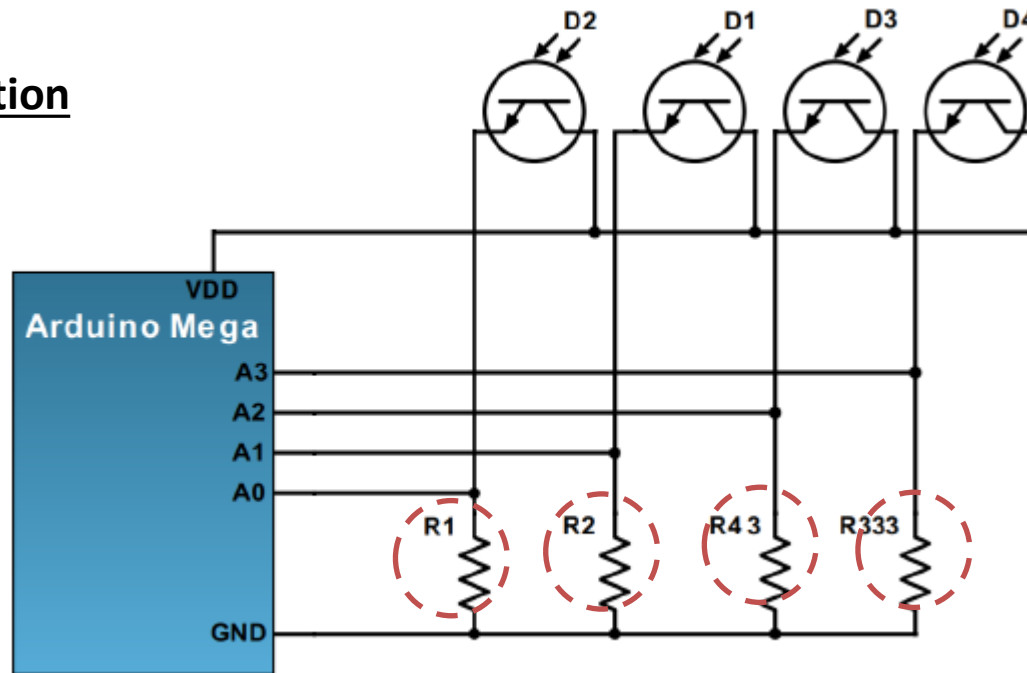
What the subsystem does:

1. To detect and extinguish a fire placed on the top of an obstacle.
2. The fire was in the form of a LED with a thermistor attached.
3. It was detected with phototransistors and then extinguished with a fan from a distance of **20** cm.

Initial Design



Final Implementation



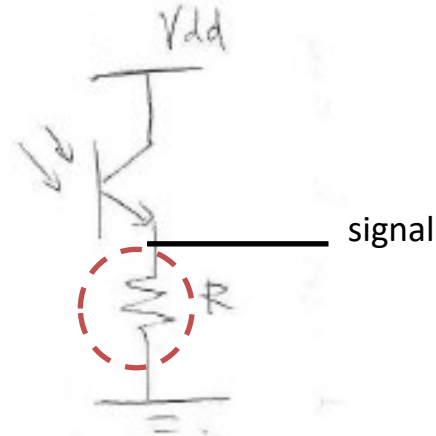
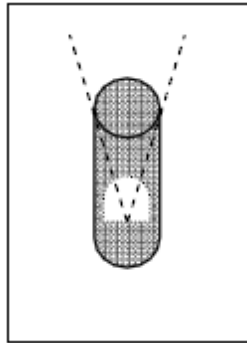
How the subsystem was designed:

1. The firefighting subsystem, to be called 'turret' subsequently, of the platform operates almost entirely separate from the other subsystems. **The firefighting turret** situated at the front of the platform scans an arc covering the front 120° of the platform, consisting of two phototransistors offset 40° to each other and a fan directed between them.
2. Selecting the **voltage divider resistor** for accurate light sensing proved challenging as resistor values higher than $5K\Omega$ would give maximum ADC reading for a wide angle pointing at the lamp. **$5K\Omega$** allows the ADC to be near to maximum when the light is in direct line to the transistor and extremely close.
3. During normal operation the **turret scans back and forth**, until either phototransistor sensor reads an illumination value above **a set threshold**, at which point the turret will alter or maintain the direction to place the light in between the two phototransistor headings. When the **light is sensed to be between** the phototransistors, the fan is activated that extinguishes the 'fire'.

Design 4 (Sophisticated)

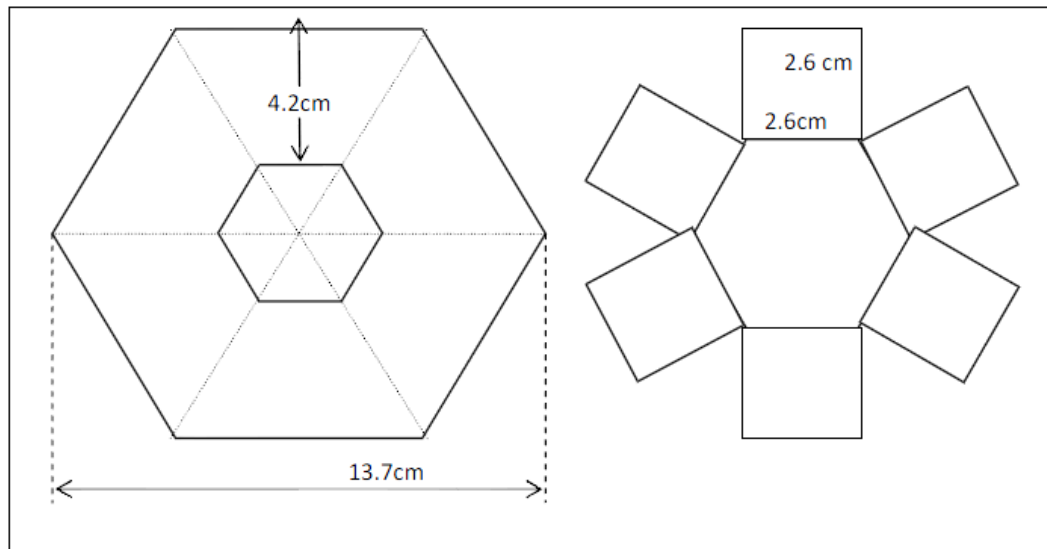
What the subsystem does:

- the direction/distance of fire



The circuit operates in active or switching mode?

The view angle of PT is narrowed by a cover



How the subsystem was designed:

Experiment result of the above circuit. The result was recorded in a room environment.

Output voltage (V)		Distance (cm)												
		10	15	20	25	30	35	40	45	50	55	60	65	70
Resistor (Ω)	470	0.37	0.37	0.35	0.33	0.31	0.28	0.19	0.12	0.07	0.04	0.03	0.01	0.01
	220k	4.78	4.76	4.74	4.68	4.63	4.37	3.97	3.36	3.01	2.48	2.12	1.66	1.48
	560k	4.88	4.88	4.85	4.83	4.82	4.70	4.44	3.60	3.00	2.90	2.43	2.10	1.90

Output voltage for different light environment using a resistor of 560 k Ω .

Output voltage (V)		Distance (cm)												
		10	15	20	25	30	35	40	45	50	55	60	65	70
Lighting	Dark	4.88	4.88	4.85	4.83	4.82	4.70	4.44	3.60	3.00	2.90	2.43	2.10	1.90
	Light	4.88	4.87	4.86	4.84	4.82	4.80	4.54	4.38	4.07	3.99	3.86	3.90	3.88

Output voltage for different light environment using the prototype cover.

Output voltage (V)		Distance (cm)												
		10	15	20	25	30	35	40	45	50	55	60	65	70
Lighting	Dark	4.88	4.88	4.85	4.83	4.82	4.70	4.44	3.60	3.00	2.90	2.43	2.10	1.90
	Light	4.88	4.87	4.86	4.84	4.82	4.80	4.70	4.10	3.40	2.90	2.60	2.30	2.14

Resultant output voltage of the different angles.

Output voltage (V)		Angle (degrees)					
		0	10	20	30	40	50
Distance (cm)	40	4.84	4.83	4.82	4.82	2.1	1.9
	50	4.82	4.79	4.45	4.72	1.8	1.8
	60	4.82	4.8	4.1	3.6	1.7	1.7

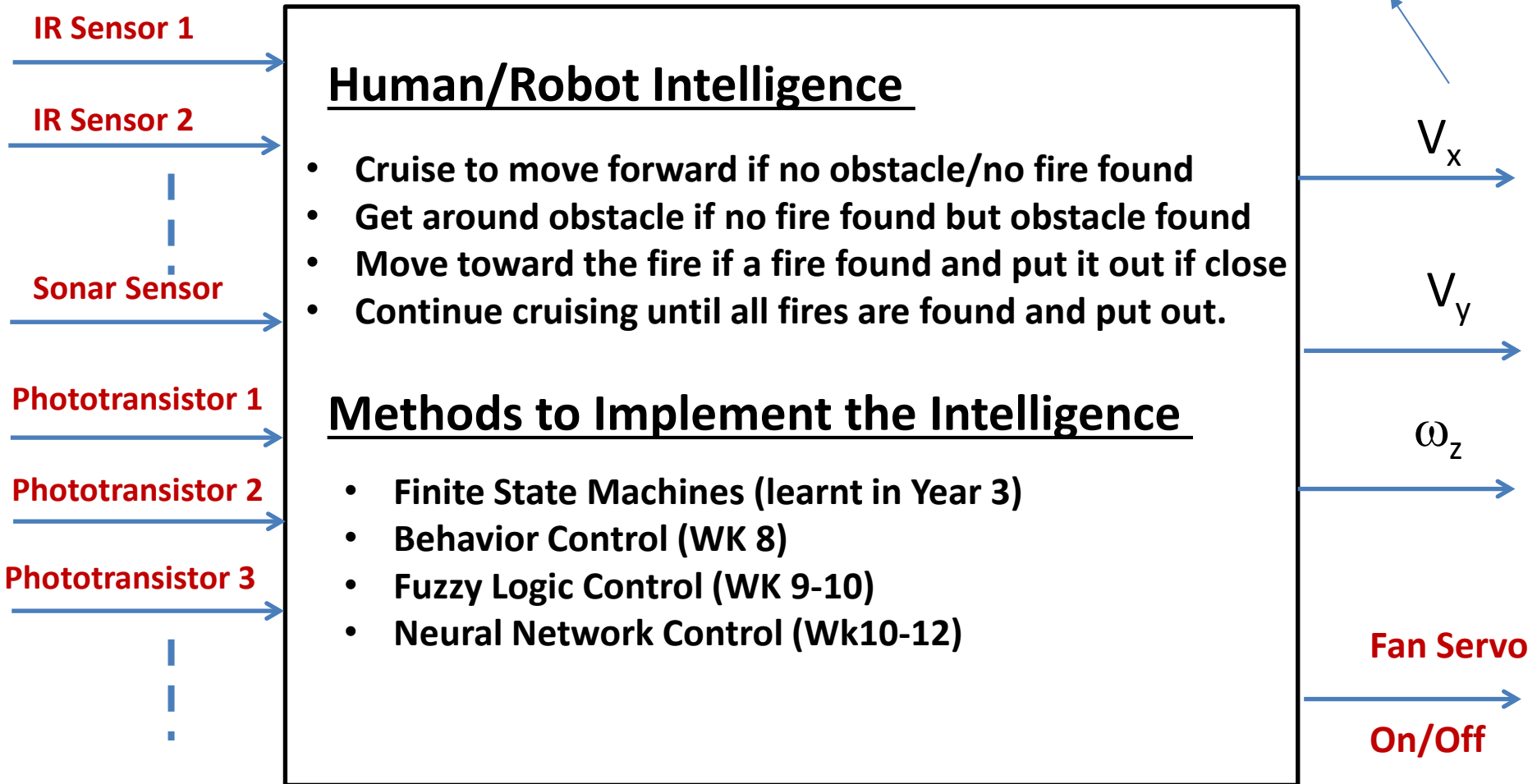
How the subsystem works (measurement):

1. Initialise and set up
2. Read in the voltages from six sensors through ADC.
3. Sort the numbers from smallest to largest (or bubble sort), which makes the later steps easier.
4. Calculate the average measured voltage using the first 3 sorted numbers (3 smallest values). This average would tell the approximate lighting environment the robot is in.
5. Based on the average value, threshold voltage is calculated ($V_{th} = V_{env} + 0.8V$).
6. Compare the threshold voltage and sorted sensor voltages in sequence. The first sensor that has larger voltage than the threshold is taken for the output. This way, even if the robot sees more than one candles , it would deal with one flame at a time.
7. Generate an output (or the direction of the candle) to the main programme.

5. Robot Intelligence System (With respect to Project 2)

- How to get the robot intelligence implemented

Robot motion can be defined by the heading direction and speed of the robot movement.



6. Closed-Loop Control System

Design 1

(With Respect to Project 1)

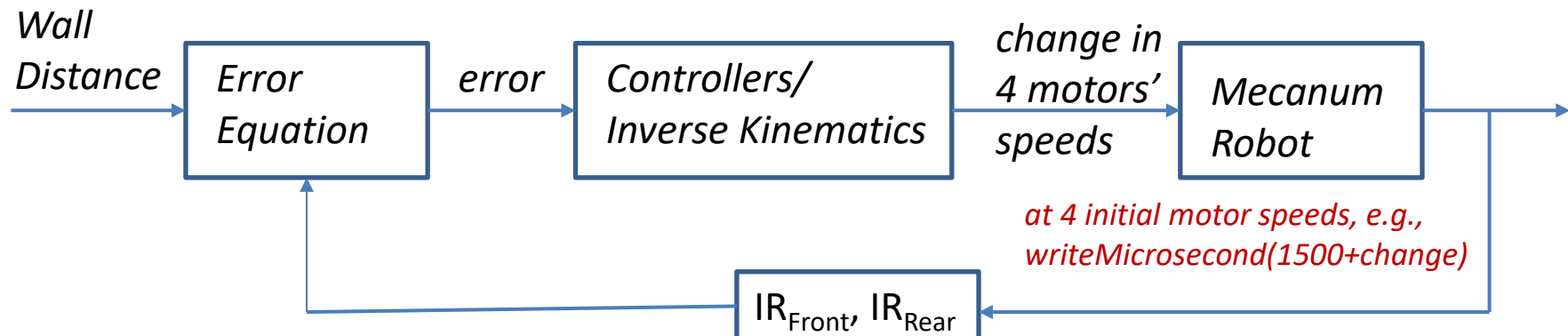
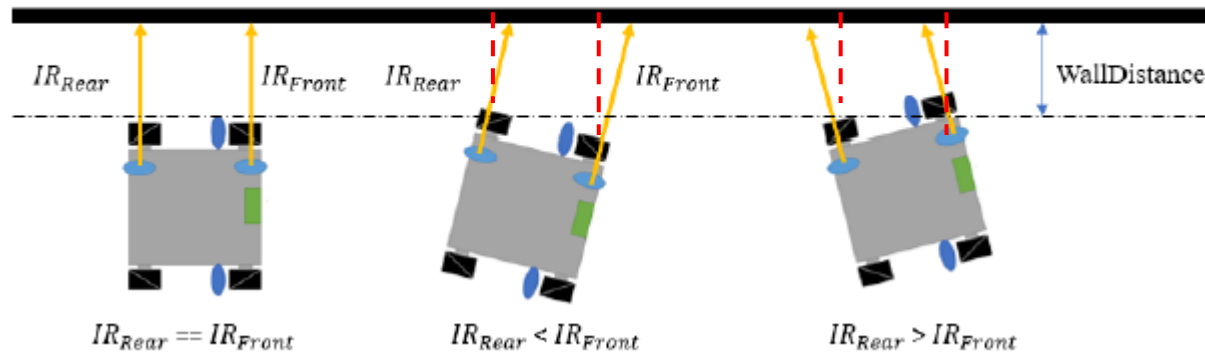
What the subsystem does:

- the robot tracks along the walls of the arena as closely as possible.
- the robot keeps the same orientation (or yaw angle).

How the subsystem was designed:

$$\text{error} = w_1 \text{error 1} + w_2 \text{error 2}$$

$$\text{error} = IR_{Front} - IR_{Rear} + (IR_{Front} - \text{WallDistance})$$



How the subsystem was implemented:

Theoretically, I action is for zero-steady error, e_{ss}

- Two control methods were explored: P-control and **PI-control**.
- The error for the controller was calculated in two parts. **The 1st part is to keep the robot parallel to the wall and the 2nd part is to keep the robot a set distance to the wall.**
- The two short range IR sensors were used and placed on the left-hand side of the robot.
- The error calculation tries to keep the robot parallel to the wall, and at a predetermined distance from the wall.
- The controller was implemented in Arduino microcontroller. After repeated testing *it was found that the integral part of the controller was causing problems due to integral wind up, meaning accurate set point tracking could not be achieved.* Subsequently the platform would oscillate along the wall edge. By removing the integral term and using a purely proportional controller, greater accuracy was achieved when following the wall.
- The controller then outputs **the change to the motor speed** of either the left or right motors to track a zero set-point value, which means that the robot is travelling parallel to the arena wall and at the correct distance from the wall.

Design 2

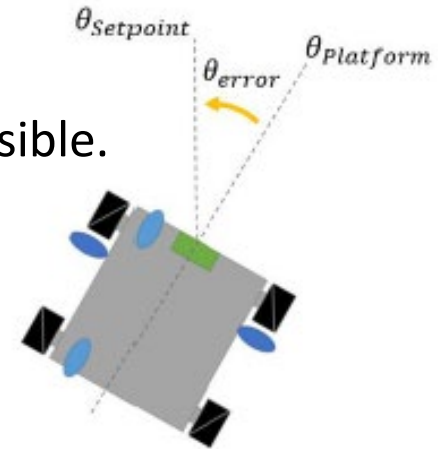
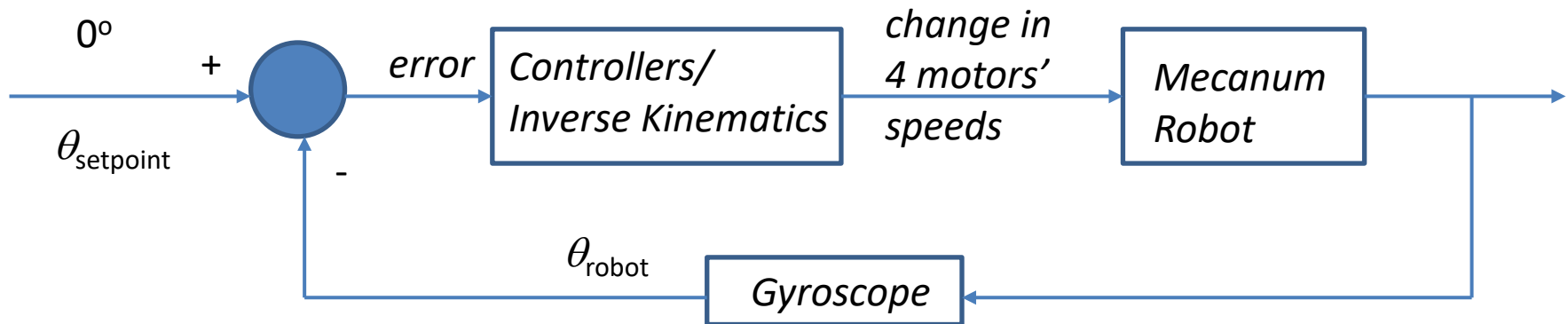
What the subsystem does:

- the robot tracks along the walls of the arena as closely as possible.
- the robot keeps the same orientation (or yaw angle).

How the subsystem was designed:

- To keep the robot going straight, the gyroscope was used to give the angle of orientation of the robot in yaw.
- A setpoint angle of 0° is then set and the robot attempts to track about 0° using a proportional controller.
- The calculation of the error for the controller is as follows,

$$\text{error} = \theta_{\text{SetPoint}} - \theta_{\text{Platform}}$$



- When the robot tracks another wall, it can turn a right angle at the corner, $\theta_{\text{setpoint}} = 90^\circ$, and moves forward.

Design 3: Design 1 and 2 are combined so as to define the 3-term Error equation.

The End



THE UNIVERSITY OF
AUCKLAND
Te Whare Wananga o Tamaki Makaurau
NEW ZEALAND

ENGINEERING